

目录

1	引言	1
2	相关工作	2
2.1	大模型与多模态推理服务	2
2.2	分离式推理	2
2.3	扩散模型并行推理	3
2.4	异构部署	3
2.5	推理缓存技术	4
3	系统设计	4
3.1	系统整体架构	4
3.2	Stage 分离与异构部署	4
3.2.1	Stage 分离设计	4
3.2.2	异构硬件亲和性匹配	5
3.2.3	设备拓扑	5
3.3	跨硬件流水线调度	6
3.3.1	任务队列管理	6
3.3.2	动态批处理策略	6
3.3.3	异步流水线调度	6
3.4	分层启动缓存	7
3.4.1	三级缓存结构	7
3.4.2	缓存预加载与淘汰策略	8
3.5	结果聚合模块	8
4	实验评估	8
4.1	实验设置	8
5	结论	8

一种基于 STAGE 分离和异构部署的多模态推理优化系统

高忠山、曾立

【摘要】

-

【关键词】

-

1 引言

多模态大模型在图像生成领域的主要任务有文生图 (T2I), 图生图 (I2I), 图文编辑 (TI2I), 其规模化应用对推理系统提出了高并发、低延迟的严峻挑战。当前主流的多模态图像生成模型如 Qwen-Image^[1]采用 VAE 编码器 (I2I, TI2I) -大语言模型特征编码-MMDiT 扩散去噪-VAE 解码器的复合架构, 推理流程涉及参数规模、计算特征迥异的多个环节。然而, 现有推理服务系统如 vLLM-Omni^[2]虽已实现对自回归多模态模型的 stage 分离推理, 但普遍未实现 stage 深度分离与异构硬件精准匹配, 存在显著技术缺陷, 无法满足高并发、低延迟的规模化推理需求。由此导致三大核心问题: (1) 异构硬件资源利用率低, 不同算力等级的 AI 处理器与 CPU 间协同效率低下; (2) 推理各阶段耦合度高, 资源争抢严重, 端到端延迟难以控制; (3) 大模型权重加载耗时过长, 冷启动延迟无法满足高频次推理服务需求。

针对上述问题, 本文提出一种基于 stage 分离和异构部署的多模态推理优化系统。主要贡献如下:

1. 将多模态模型的文生图推理 pipeline 拆分为文本预处理、LLM 特征编码、MMDiT Diffusion 生成、VAE Decoder 四个功能独立、边界清晰的 stage, 并针对昇腾 NPU 多型号 (310P/910C/910B 等) + CPU 的异构硬件环境, 建立基于算力特征、参数规模、耗时占比的硬件亲和性匹配规则, 实现各 stage 与异构硬件的精准适配。

2. 设计跨硬件异步流水线调度机制，为每个 stage 维护专属任务队列并结合动态批处理策略，消除阶段间同步等待，显著提升异构硬件并行效率与推理吞吐量。
3. 构建适配 stage 分离架构的三级分层缓存（模型权重缓存、中间特征缓存、推理参数缓存），实现阶段级缓存预加载与淘汰更新，大幅缩短模型冷启动时间。

2 相关工作

2.1 大模型与多模态推理服务

以大语言模型 (LLM) 高效推理服务为核心，vLLM^[3] 提出 PagedAttention 机制实现 KV-cache 内存近零浪费管理，显著提升 LLM 推理吞吐量。Miao 等^[4] 从算法创新与系统优化两个维度系统综述了生成式 LLM 推理服务的效率优化方法，覆盖解码算法、模型压缩、并行计算、内存管理、请求调度等关键方向。SARATHI^[5] 通过 chunked-prefill 与 decode-maximal batching 策略，将计算密集的 prefill 与内存密集的 decode 阶段统一调度，有效消除流水线气泡。在多模态服务方向，vLLM-Omni^[2] 提出面向 Any-to-Any 多模态模型的完全分离式服务架构。对 Thinker-Talker（如 Qwen2.5-Omni, Qwen3-Omni 等）、AR+DiT（如 GLM-Image 等），AR+ 专用生成器（如 BAGEL 等），通过 stage 级别抽象实现跨模态组件的独立调度与资源分配。

2.2 分离式推理

分离式推理 (Disaggregated Inference) 将 LLM 推理的 prefill（计算密集）与 decode（内存密集）两阶段拆分至不同硬件，实现独立资源分配与并行度优化。Splitwise^[6] 率先论证了 prefill/decode 阶段分离的经济性与可行性，通过将两个阶段部署至不同 GPU 集群实现 1.4 倍吞吐提升与 20% 成本降低。DistServe^[7] 进一步将分离策略与 SLO 约束协同优化，消除 prefill-decode 干扰。TetriInfer^[8] 提出两级调度 + 资源预测的分离式架构，降低混合负载下的推理干扰。现有分离式推理工作对 DiT 文生图流水线（文本编码 → 扩散去噪 → VAE 解码）多阶段分离的异构推理部署涉及较少。本工作将异构分离式推理范式拓展至扩散模型领域，提出面向文生图流水线的四阶段深度分离方案。

2.3 扩散模型并行推理

扩散模型的高分辨率图像生成面临巨大计算开销。DistriFusion^[9]提出 *displaced patch parallelism*，利用相邻扩散步骤特征图的高相似性，通过复用前序步骤的 *stale feature map* 实现通信与计算异步重叠实现了最高 6.1 倍加速。PipeFusion^[10]设计 *patch* 级流水线并行策略，将 DiT 模型层与图像 *patch* 分配至多 GPU，复用一步 *stale* 特征降低通信开销，配合参数分布存储提升内存效率。xDiT^[11]整合序列并行、PipeFusion 流水线并行与 CFG 并行，构建可灵活组合的混合并行推理引擎，首次在以太网互联 GPU 集群上展示 DiT 可扩展性。DiT-Serve^[12]则从跨请求批处理优化角度出发，提出 *step-level batching*，在每个 *denoising step* 边界抢占和交换请求以消除异构请求间的等待气泡，并设计 *Brick Attention* 将不同上下文长度的请求 *binpack* 到多组独立 *ring* 上，减少 *padding* 开销。TridentServe^[13]针对扩散 *pipeline* 内在的 *stage* 异质性与请求异质性，提出动态 *stage* 级服务范式，通过自动 *placement plan* 和 *dispatch plan* 实现各 *stage* 的细粒度资源分配，消除静态部署导致的资源浪费。上述并行推理工作主要聚焦于同构 GPU 环境下的模型内部加速或批处理调度优化，TridentServe 虽实现了 *stage* 级动态分配但基于同构 GPU 集群，尚未覆盖异构加速器（如昇腾 NPU 多型号）的算力梯度差异。本工作将 *stage* 分离与异构硬件亲和性匹配相结合，面向昇腾 310P/910C/910B 等多型号 NPU+CPU 环境，提出四阶段深度分离与精准异构部署方案。

2.4 异构部署

异构计算环境下 CPU 与 GPU/加速器的协同部署是降低推理成本的重要路径。HeteGen^[14]提出 CPU+GPU 异构张量并行框架，通过异步重叠通信与计算缓解 I/O 瓶颈，实现资源受限设备上的高效 LLM 推理。SiPipe^[15]利用未充分利用的 CPU 资源卸载辅助计算与通信任务，通过 CPU *sampling*、*token-safe* 执行模型与结构感知传输三项技术缓解流水线气泡，在多 GPU 流水线并行部署中提升 GPU 利用率达 23%。Hybrid SD^[16]将 Stable Diffusion 的早期扩散步骤部署于云端大模型实现语义规划，后期步骤部署于边缘端小模型完成视觉细节精炼，实现边缘-云协同扩散推理。现有异构部署方案止于 CPU-GPU/云端-边缘的二元划分，尚未针对昇腾 NPU 多型号（310P/910C/910B 等）的算力梯度差异做逐 *stage* 的硬件亲和性精准匹配。本工作首次提出面向四阶段异构硬件的 *stage*-硬件亲和性匹配方法。

2.5 推理缓存技术

缓存技术是缩短模型启动延迟、提升推理效率的关键手段。LMCache^[17]提出层次化 KV 缓存方案，支持 GPU 显存 ↔ CPU 内存 ↔ 磁盘 ↔ 远程存储的多层级缓存迁移与跨引擎复用，在 prefix caching 场景实现最高 15 倍吞吐提升。IMPRESS^[18]构建基于重要性的多级前缀 KV 存储系统，根据访问频率动态调整缓存层级，实现高命中率下的存储成本优化。PRESERVE 通过模型权重与 KV Cache 的预取实现通信与计算重叠，缩短冷启动延迟。FlashPS^[19]面向图像编辑场景，通过缓存和复用未编辑区域的中间 activations 跳过冗余计算，并设计 bubble-free pipeline 将缓存加载与计算流水线重叠，是扩散模型场景下结合缓存与计算调度的代表性工作。现有缓存方案几乎全部聚焦于 KV-cache 的层次化管理与跨请求复用，FlashPS 虽涉及扩散模型的中间特征缓存但面向图像编辑的特殊 mask 场景，尚未出现适配通用文生图 stage 分离架构的模型权重 + 中间特征 + 推理参数三级缓存设计。本工作提出了与四阶段分离推理深度耦合的三级分层缓存机制，填补了这一空白。

3 系统设计

3.1 系统整体架构

本系统由四个核心模块组成：stage 异构部署模块、跨硬件流水线调度模块、分层启动缓存模块、结果聚合模块。各模块数据互通、协同工作，形成闭环式推理优化架构。

3.2 Stage 分离与异构部署

3.2.1 Stage 分离设计

以 Qwen-Image 多模态大模型为例，针对多模态文生图推理任务场景，我们将模型推理流程拆分为 4 个边界清晰功能独立的 stage，具体如下：

- 文本预处理 stage: 负责多模态中文本预处理与嵌入编码，将用户输入的文本提示词转换为初始特征向量，为后续文本特征深度提取提供基础；
- LLM 特征编码 stage: -负责对文本预处理 stage 输出的初始特征向量

进行深度文本特征提取与编码，进一步优化文本信息向特征向量的转换效果；

- **MMDiT Diffusion 生成 stage:** -负责潜空间图像特征去噪过程，为多模态推理流程的计算密集型环节；
- **VAE 解码 stage:** -负责多模态中的图像重建，输出最终多模态文生图效果。

3.2.2 异构硬件亲和性匹配

基于多模态各 stage 的算力需求、参数规模、耗时占比、显存阈值，结合多模态双模态特性，精准分配至对应硬件，具体部署逻辑如下：

- **文本预处理 stage**→ 通用处理器（例如 **CPU**）：多模态中的轻量文本预处理与初始编码环节，例如参数规模约 84M，耗时占比 < 2%，充分利用通用处理器资源，避免浪费 AI 处理器（例如昇腾 NPU）算力，适配多模态轻量文本处理需求；
- **LLM 特征编码 stage**→ 中端 AI 处理器（例如昇腾 **NPU-310P**）：多模态中的轻量文本编码环节，例如参数规模 7B，耗时占比 < 3%，适配中端 AI 处理器算力、24GB 显存，释放高端 AI 处理器资源，适配多模态轻量文本处理需求；
- **MMDiT Diffusion 生成 stage**→ 高端 AI 处理器（例如昇腾 **NPU-910C**）：多模态图文双模态融合及核心生成环节，例如参数规模 20B，耗时占比 60%-80%，充分发挥高端 AI 处理器并行计算优势、64GB 大显存特点，满足多模态密集型计算需求；
- **VAE 解码 stage**→ 对应 AI 处理器（例如昇腾 **NPU-910B**）：多模态中的图像重建环节，例如参数规模约 100M，耗时占比 10%-20%，适配 32GB 显存，与核心生成 stage 并行执行，提升多模态推理整体效率；

3.2.3 设备拓扑

-

3.3 跨硬件流水线调度

3.3.1 任务队列管理

系统为每个 stage 维护独立的输入队列与输出缓冲，形成四级解耦的任务队列架构：CPU(文本预处理)→310P(LLM 编码)→910C(MMDiT 去噪)→910B(VAE 解码)。队列条目包含请求 ID、数据引用指针、中间特征引用及优先级标签。

为防止上下游处理速率失配导致内存膨胀，引入自适应背压机制：设各 stage 输入队列安全阈值 Q_{safe} 与上限 Q_{max} ，当队列深度 d 满足 $Q_{\text{safe}} < d \leq Q_{\text{max}}$ 时，上游分发间隔按比例 $\frac{d-Q_{\text{safe}}}{Q_{\text{max}}-Q_{\text{safe}}}$ 递增；当 $d > Q_{\text{max}}$ 时暂停投递。优先级调度采用两级策略：下游 stage 的输出队列优先级高于上游以加速端到尾端推进；短提示词优先于长提示词以减少排队延迟。

3.3.2 动态批处理策略

系统在 MMDiT Diffusion stage 上实现 step 级连续批处理：每个去噪步 (denoising step) 结束后，已完成全部步数的请求立即退出 batch，其潜空间张量被转发至 VAE 解码队列，释放的显存被回收；同时输入队列中的新请求在一步内加入下一 batch。该机制将 batch 空闲率从静态方案的约 40% 降至 5% 以下，与 Orca^[20]在 LLM 推理中提出的连续批处理、DiT-Serve^[12]在 DiT 推理中提出的 step-level batching 思路一致。

各 stage 的批处理大小 B_s 根据三个约束实时计算并取最小值： $B_s^{\text{queue}} = \min(B_s^{\text{max}}, n_{\text{pending}})$ (队列积压)、 $B_s^{\text{mem}} = \lfloor (M_{\text{total}} - M_{\text{model}}) / M_{\text{per_req}} \rfloor$ (显存约束)、 $B_s^{\text{slo}} = \arg \max_k \{\text{Latency}(k) \leq T_{\text{slo}}\}$ (SLO 约束)。各 stage 因硬件差异采用差异化上限：CPU 文本预处理 $B_{\text{text}} \leq 64$ ，310P 端 LLM 编码 $B_{\text{llm}} \leq 8$ (24GB 显存)，910C 端 MMDiT $B_{\text{dit}} \leq 4$ (1024×1024 单请求 ~16GB)，910B 端 VAE $B_{\text{vae}} \leq 16$ 。

3.3.3 异步流水线调度

系统采用全异步流水线执行模型：stage 完成当前任务后立即将输出写入下游输入队列并取回下一批任务，无需等待下游确认。跨硬件数据传输采用双缓冲机制——每个 stage 输出端维护写入缓冲与传输缓冲，写入完成后角色互换，DMA 传输与下一请求的计算完全重叠。具体传输路径：CPU→310P 走 PCIe 4.0 (文本嵌入 ~64KB，延迟约 2μs)，310P→910C 走 HCCS (编码

特征 $\sim 100\text{MB}$ ，延迟约 2ms ），910C $\rightarrow$ 910B 走 HCCS（潜空间张量 $\sim 8\text{MB}$ ，延迟约 0.16ms ）。

系统采用流水线滑窗控制并发请求数上限 $K_{\max} = 16$ ，按 MMDiT 批大小上限（4）与各 stage 处理延迟比（ $\approx 1:2:50:10$ ）归一化分配窗口配额，将峰值内存控制在理论最大值的 65% 以内。

3.4 分层启动缓存

3.4.1 三级缓存结构

构建与 stage 分离架构深度适配的三级分层缓存：

1. 模型权重缓存：缓存各 stage 对应的模型主干 FP16 权重。文本预处理 $\sim 168\text{MB}$ （84M 参数），LLM 编码 $\sim 14\text{GB}$ （7B 参数，加载至 310P 24GB HBM），MMDiT $\sim 40\text{GB}$ （20B 参数，加载至 910C 64GB HBM），VAE $\sim 200\text{MB}$ （100M 参数，加载至 910B 32GB HBM）。权重按 stage 粒度独立加载与卸载，四设备并行加载将冷启动时间从全量串行加载的 $\sim 60\text{s}$ 压缩至 $\sim 12\text{s}$ 。
2. 中间特征缓存：缓存前序 stage 的输出特征张量，避免重复执行上游推理。核心条目包括：（a）文本编码特征——提示词经 LLM 编码后的条件向量，OpenPrompt 等预置风格模板命中率可达 90% 以上；（b）VAE 图像编码特征——输入图像的潜空间表示，复用于相同图像的多次编辑；（c）风格 LoRA 嵌入——高频风格权重驻留在 NPU HBM 中。特征缓存采用主机内存（DDR）+ 设备内存（HBM）两级存储层次。

在 MMDiT Diffusion stage 内部，本系统进一步引入 CacheDiT^[21]的块级特征缓存技术以加速逐 step 去噪过程。具体集成三项子策略：（a）**DBCACHE**（Dual Block Cache）：缓存在相邻去噪步间输出变化趋于稳定的前 n 个 Transformer block（ F_n ）和末 n 个 block（ B_n ）的中间激活值，在后续 step 中复用缓存值跳过这些 block 的重计算，配置为 F8B8 时可将去噪步计算量降低约 40%；（b）**FBCACHE**（First Block Cache）：轻量级变体，仅缓存第一个 block 的输出，适用于 batch 内负载不均时的选择性加速，计算开销约为 DBCache 的 1/8；（c）**CACHE CFG**：在分类器自由引导（Classifier-Free Guidance）推理中，将无条件分支的预测结果单独缓存，相邻 step 间无条件分支的特征漂移小于条件分支，实现额外 1.3—2.0 $\times$ 的 CFG 加速。

3. 推理参数缓存：缓存采样器类型、CFG scale、随机种子、去噪步数、输出分辨率等运行时参数，以 JSON 结构化按会话 ID 索引存储于 CPU 内存。FlashPS^[19]在图像编辑场景中采用了按步状态缓存的思路，本工作将其扩展至通用文生图全流程的多级联合缓存。

3.4.2 缓存预加载与淘汰策略

系统启动时，四个设备的权重并行加载（CPU<1s, 310P~5s, 910C~12s, 910B<1s）。中间特征缓存在系统初始化后根据历史 trace 预计算 Top-200 高频提示词模板的文本编码特征（~3s），CacheDiT 的 FBCache 在首个请求的前几步去噪中自动完成块级特征预热。

淘汰策略上，权重缓存采用基于 stage 滑动窗口利用率的 LRU：当某设备剩余显存低于 10% 时卸载利用率最低的非活跃 stage 权重至主机内存，重载速度约为初始加载的 3—5 倍。中间特征缓存沿用 LMCache^[17]的层级迁移思路：设备内存中低频（访问计数 $f < 3$ ）且最久未用的条目迁移至主机内存，主机内存中超过 TTL（ $T_{\text{TTL}} = 30\text{min}$ ）的条目彻底删除，维持 HBM 命中率 $\geq 92\%$ 。推理参数缓存采用会话级 TTL（1h）惰性淘汰。

3.5 结果聚合模块

-

4 实验评估

4.1 实验设置

硬件环境 CPU: Kunpeng-920; 昇腾 NPU-310P (显存 24GB); 昇腾 NPU-910C (显存 64GB); 昇腾 NPU-910B (显存 32GB)。

5 结论

-

参考文献

- [1] WU C, LI J, ZHOU J, et al. Qwen-Image Technical Report[EB/OL]. arXiv. 2025 [2026-04-27]. <http://arxiv.org/abs/2508.02324>.
- [2] YIN P, ZHU J, GAO H, et al. vLLM-Omni: Fully Disaggregated Serving for Any-to-Any Multimodal Models[EB/OL]. arXiv. 2026 [2026-04-27]. <http://arxiv.org/abs/2602.02204>.
- [3] KWON W, LI Z, ZHUANG S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention[EB/OL]. arXiv. 2023 [2026-04-27]. <http://arxiv.org/abs/2309.06180>.
- [4] MIAO X, OLIARO G, ZHANG Z, et al. Towards Efficient Generative Large Language Model Serving: A Survey from Algorithms to Systems[J]. ACM Computing Surveys, 2026, 58(1): 1-37. DOI: [10.1145/3754448](https://doi.org/10.1145/3754448).
- [5] AGRAWAL A, PANWAR A, MOHAN J, et al. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills[EB/OL]. arXiv. 2023 [2026-04-27]. <http://arxiv.org/abs/2308.16369>.
- [6] PATEL P, CHOUBE E, ZHANG C, et al. Splitwise: Efficient generative LLM inference using phase splitting[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2311.18677>.
- [7] ZHONG Y, LIU S, CHEN J, et al. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2401.09670>.
- [8] HU C, HUANG H, XU L, et al. Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2401.11181>.
- [9] LI M, CAI T, CAO J, et al. DistriFusion: Distributed Parallel Inference for High-Resolution Diffusion Models[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2402.19481>.
- [10] FANG J, PAN J, LI A, et al. PipeFusion: Patch-level Pipeline Parallelism for Diffusion Transformers Inference[EB/OL]. arXiv. 2025 [2026-04-27]. <http://arxiv.org/abs/2405.14430>.

- [11] FANG J, PAN J, SUN X, et al. xDiT: an Inference Engine for Diffusion Transformers (DiTs) with Massive Parallelism[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2411.01738>.
- [12] LUO M, HAO A, YAN Z, et al. DiT-Serve: An Efficient Serving Engine for Diffusion Transformers[J]. 2025.
- [13] XIA Y, FU F, YUAN H, et al. TridentServe: A Stage-level Serving System for Diffusion Pipelines[EB/OL]. arXiv. 2025 [2026-04-29]. <http://arxiv.org/abs/2510.02838>.
- [14] ZHAO X, JIA B, ZHOU H, et al. HeteGen: Heterogeneous Parallel Inference for Large Language Models on Resource-Constrained Devices[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2403.01164>.
- [15] HE Y, ZHAO B, CAO Z. SiPipe: Bridging the CPU-GPU Utilization Gap for Efficient Pipeline-Parallel LLM Inference[EB/OL]. arXiv. 2025 [2026-04-27]. <http://arxiv.org/abs/2506.22033>.
- [16] YAN C, LIU S, LIU H, et al. Hybrid SD: Edge-Cloud Collaborative Inference for Stable Diffusion Models[EB/OL]. arXiv. 2024 [2026-04-27]. <http://arxiv.org/abs/2408.06646>.
- [17] LIU Y, CHENG Y, YAO J, et al. LMCACHE: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference[EB/OL]. arXiv. 2025 [2026-04-27]. <http://arxiv.org/abs/2510.09665>.
- [18] CHEN W, HE S, QU H, et al. IMPRESS: An Importance-Informed Multi-Tier Prefix KV Storage System for Large Language Model Inference[C]//. 2025: 187-201.
- [19] JIANG X, LI S, YANG L, et al. FlashPS: Efficient Generative Image Editing with Mask-aware Caching and Scheduling[C]//EUROSYS '26: Proceedings of the 21st European Conference on Computer Systems. New York, NY, USA: Association for Computing Machinery, 2026: 2109-2125. DOI: [10.1145/3767295.3769379](https://doi.org/10.1145/3767295.3769379).
- [20] YU G I, JEONG J, KIM G W, et al. Orca: A Distributed Serving System for Transformer-Based Generative Models[C]//OSDI '22: Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation. Carlsbad, CA, USA: USENIX Association, 2022: 521-538.

- [21] Vipshop.com. CacheDiT: A Training-free and Easy-to-use Cache Acceleration Toolbox for Diffusion Transformers[EB/OL]. 2025 [2026-04-29]. <https://github.com/vipshop/cache-dit>.